

Intro to Machine Learning	• BA4
1 Notations & Core Concepts	
$x_i \in \mathbb{R}^D$ — sample (column vector, D features) $y_i \in \mathbb{R}$ (regression) or $\{0, 1\}^C$ (classification) $X \in \mathbb{R}^{N \times D}$, $y \in \mathbb{R}^N$ — dataset $\hat{y}_i$ — prediction for sample i $w^\star$ — optimal parameters	
Empirical Risk	
$R(w) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i) \Rightarrow \min_w R(w)$	

Gradient Descent
$w_k \leftarrow w_{k-1} - \eta \nabla_w R(w_{k-1})$ Stop when $R(w_k) - R(w_{k-1}) < \delta$ or $\ w_k - w_{k-1}\ < \delta$.

Useful Identities
$\nabla_b (a^T b) = a \quad \nabla_a \ a\ ^2 = 2a \quad \ a\ ^2 = a^T a$

2 Preprocessing

Normalization

Why: avoid one feature dominating due to scale. Apply **always** before distance-based methods (KNN, K-Means) and improves convergence for gradient-based methods.

Methods		
Method	Formula	Range
Z-score	$\frac{x^{(d)} - \mu_d}{\sigma_d}$	$\mu=0, \sigma=1$
Min-max	$\frac{x^{(d)} - \min_d}{\max_d - \min_d}$	$[0, 1]$
Max norm	$\frac{x^{(d)}}{\max_d}$	$[-1, 1]$

Bias term: append 1 to x_i so $D \leftarrow D + 1$.

Missing Data

- Drop the sample (loses info)
- Fill with global constant
- Fill with feature mean
- Fill with class-conditioned mean (best)

Noise / Binning

Sort $\rightarrow$ partition into bins (equal width or equal frequency) $\rightarrow$ replace each value by bin mean/median.

Imbalanced Data

Problem: 99% negative class $\Rightarrow$ predicting always negative gives 99% accuracy but misses all positives.

Sampling Methods
Oversampling: replicate or generate (SMOTE) minority samples Undersampling: remove redundant majority samples

Cost-Sensitive Reweighting
$R = \frac{1}{N} \sum_i \beta_i \ell(\hat{y}_i, y_i), \quad \beta_i = 1 - \frac{N_k}{N}$ N_k = size of class k of sample i. Smaller class $\Rightarrow$ larger β_i.

Text: Bag of Words (BoW)

Build vocabulary of D words. For each document: count occurrences $\Rightarrow$ normalize by total words $\Rightarrow x_i \in \mathbb{R}^D$.

Limits: no word order; frequent stopwords dominate; sparse vectors.

Images: Visual BoW

Extract patches $\rightarrow$ cluster patches into D visual words $\rightarrow$ count occurrences per image $\rightarrow$ normalize $\Rightarrow x_i \in \mathbb{R}^D$.

3 Linear Regression

When: output is continuous; relationship is approximately linear.

Output: $\hat{y}_i \in \mathbb{R}$.

Model & Loss (MSE)
$\hat{y}_i = w^T x_i \quad R(w) = \frac{1}{N} \sum_{i=1}^N (w^T x_i - y_i)^2$

Closed-Form Solution
$\nabla_w R = 0 \Rightarrow w^\star = (X^T X)^{-1} X^T y$ If $X^T X$ not invertible: $w^\star = X^\dagger y$ (pseudo-inverse). Multi-output ($C > 1$): $W^\star = X^\dagger Y$

Prediction
$\hat{y} = w^{\star T} x \quad (C=1) \quad \hat{y} = W^{\star T} x \quad (C>1)$

Pros: exact closed-form, fast, interpretable. *Cons:* only linear relationships.

4 Logistic Regression (Binary)

When: binary classification ($y_i \in \{0, 1\}$); approximately linearly separable.

Output: $\hat{y}_i = p(\text{class} = 1 \mid x_i) \in (0, 1)$.

Model
$\hat{y}_i = \sigma(w^T x_i) = \frac{1}{1 + e^{-w^T x_i}}$

Binary Cross-Entropy Loss
$R(w) = - \sum_{i=1}^N \left[y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i) \right]$

Gradient (no closed form $\Rightarrow$ GD)
$\nabla_w R(w) = \sum_{i=1}^N (\hat{y}_i - y_i) x_i$

Prediction
$\hat{y} = \begin{cases} 1 & \sigma(w^{\star T} x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$ Threshold 0.5 can be adjusted (e.g. raise for high precision).

5 Multi-Class Classification

When: $C > 2$ classes. Labels as one-hot: $y_i \in \{0, 1\}^C$.

Linear (Least Squares)

$$\min_W \sum_i \|W^T x_i - y_i\|^2 \Rightarrow W^\star = X^\dagger Y$$

$$\hat{y}_i = W^{\star T} x_i \in \mathbb{R}^C, \quad k = \arg \max_j \hat{y}_i^{(j)}$$

Softmax (Logistic)

Softmax Output
$\hat{y}^{(k)}(x) = \frac{\exp(w^{(k)T} x)}{\sum_{j=1}^C \exp(w^{(j)T} x)} \approx p(\text{class} = k \mid x)$

Multi-Class Cross-Entropy
$R(W) = - \sum_{i=1}^N \sum_{k=1}^C y_i^{(k)} \ln \hat{y}_i^{(k)}$

Gradient & Prediction
$\nabla_W R = \sum_{i=1}^N x_i (\hat{y}_i - y_i)^T \quad k = \arg \max_j \hat{y}_i^{(j)}$

6 SVM — Support Vector Machines

When: binary classification; want maximum-margin boundary; works well in high dim.

Output: $\hat{y} \in \{-1, +1\}$.

Key Concept: Margin
Distance between the hyperplane $w^T x + b = 0$ and the closest points.
$\text{margin} = \frac{2}{\ w\ } \quad (\text{maximize} \Leftrightarrow \text{minimize } \ w\)$
Support vectors: the points defining the margin.
Prediction
$\hat{y} = \text{sign}(w^{\star T} x + b)$

7 k-Nearest Neighbors (KNN)

When: non-linear decision boundaries; small dataset; no training phase needed.

Output: class label or continuous value.

Distances
Euclidean: $\sqrt{\sum_d (x_i^{(d)} - x_j^{(d)})^2}$

Classification
Find k nearest neighbors $\Rightarrow$ majority vote .

Regression
$\hat{y} = \frac{\sum_i w_i y_i}{\sum_i w_i} \quad w_i = \frac{1}{d_i} \text{ (inv. dist.)}$

Hyperparameter k
$k = 1$: very flexible, overfitting risk k large: smooth, underfitting risk Choose k by cross-validation. Use odd k for 2 classes.

Cons: no training but slow prediction $O(ND)$; sensitive to scale $\Rightarrow$ **normalize first!**; sensitive to outliers.

8 Feature Expansion & Regularization

When: data has non-linear structure but you want a linear model's tractability.

Polynomial Expansion
$\phi : \mathbb{R}^D \rightarrow \mathbb{R}^F$ maps x to higher-dim features (e.g. $[1, x_1, x_2, x_1^2, x_1 x_2, x_2^2]^T$).
$\hat{y}_i = w^T \phi(x_i), \quad w^\star = (\Phi^T \Phi)^{-1} \Phi^T y$
The model is linear in $\phi(x)$ but non-linear in x .

Cover's Theorem
A complex problem is more likely linearly separable in high-dim space (provided not densely populated).

Overfitting Risk
High-degree $\phi \Rightarrow F \gg N \Rightarrow$ overfitting. Use regularization + cross-validation to choose degree.

L2 Regularization (Ridge)

Regularized Objective

$$E(w) = \underbrace{R(w)}_{\text{fit}} + \underbrace{\lambda w^T w}_{\text{penalty}}$$

$\lambda > 0$ from cross-validation. Penalizes large weights $\Rightarrow$ smoother model.

Ridge Solution (always invertible!)

$$w^* = (\Phi^T \Phi + \lambda I_F)^{-1} \Phi^T y$$

Multi-output: $W^* = (\Phi^T \Phi + \lambda I_F)^{-1} \Phi^T Y$

9 Kernel Methods

Why: when ϕ is very high-dim or infinite-dim, computing $\phi(x)$ is infeasible. We only need dot products $\phi(x_i)^T \phi(x_j) = k(x_i, x_j)$.

Kernel Function

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j) \quad (\text{implicit } \phi)$$

Polynomial: $(x_i^T x_j + c)^d$

Kernel Matrix & Vector

$$K \in \mathbb{R}^{N \times N}, \quad K_{ij} = k(x_i, x_j) \quad (\text{symmetric, PSD})$$

$$k(X, x) = (k(x_1, x), \dots, k(x_N, x))^T \in \mathbb{R}^N$$

Kernel Ridge Regression

Reparametrize $w = \Phi^T \alpha$, solve for $\alpha \in \mathbb{R}^N$:

$$\alpha^* = (K + \lambda I_N)^{-1} y$$

$$\hat{y} = y^T (K + \lambda I)^{-1} k(X, x)$$

Multi-output: $\hat{y} = Y^T (K + \lambda I)^{-1} k(X, x)$

Kernel Selection

Choose kernel and hyperparams (d, σ, λ) by **cross-validation**.

Kernel vs explicit ϕ : kernel is $O(N^2)$ in space vs $O(NF)$

10 Neural Networks / MLP

When: large datasets; complex non-linear structure; learns features automatically.
Output: any type depending on final activation.

Architecture (Forward Pass)

Layers

$$z_{(1)} = f(W_{(1)}^T x) \quad z_{(l)} = f(W_{(l)}^T z_{(l-1)})$$

$$\hat{y} = W_{(L)}^T z_{(L-1)} \quad (\text{or softmax for classification})$$

Activation Functions

ReLU: $\max(0, z)$ — hidden layers (most common) Sigmoid: $\frac{1}{1+e^{-z}}$ — binary output Softmax: $\frac{e^{z^{(k)}}}{\sum_j e^{z^{(j)}}}$ — multi-class output

Universal Approximation

Any continuous function can be approximated by a linear combination of translated/scaled ReLU (or sigmoid) functions.

Training: Backpropagation

Loss Functions

Regression: $R(W) = \frac{1}{N} \sum_i \|\hat{y}_i - y_i\|^2$ Classification: $R(W) = -\frac{1}{N} \sum_i \sum_k y_i^{(k)} \ln \hat{y}_i^{(k)}$

Backprop Algorithm

- Forward pass: compute all $z_{(l)}$ and $\hat{y}$.
- Compute output sensitivity $\delta_{(L)}^{(k)} = \frac{\partial \ell_i}{\partial a_{(L)}^{(k)}}$.
- Propagate backward: $\delta_{(l)} \rightarrow \delta_{(l-1)}$ via chain rule.
- Gradient: $\frac{\partial \ell_i}{\partial W_{(l)}^{(k,j)}} = \delta_{(l)}^{(k)} z_{(l-1)}^{(j)}$.

SGD & Variants

Mini-Batch SGD

$$W_k \leftarrow W_{k-1} - \eta \sum_{b=1}^B \nabla \ell_{(b,k)}(W_{k-1})$$

B = batch size. Full GD: $B = N$; SGD: $B = 1$; mini-batch: $1 < B < N$.

SGD with Momentum

$$\nu \leftarrow \gamma \nu + \eta \nabla R(W) \quad W \leftarrow W - \nu$$

$\gamma > 0$ adds inertia; accelerates if gradient is consistent.

Hyperparams: learning rate η , batch size B , depth L , width M , activation.
Overfitting: use dropout, weight decay (L2), data augmentation, early stopping.

11 Dimensionality Reduction

Goal: $x_i \in \mathbb{R}^D \rightarrow y_i \in \mathbb{R}^d$ with $d \ll D$, preserving structure.

PCA — Principal Component Analysis

When: linear structure; want interpretable, fast compression.

Covariance Matrix

$$C = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})(x_i - \bar{x})^T \in \mathbb{R}^{D \times D}$$

PCA Solution

$w_{(j)} = j\text{-th eigenvector of } C \text{ (sorted } \lambda_1 \geq \dots \geq \lambda_d)$.

$$W = [w_{(1)} \cdots w_{(d)}] \in \mathbb{R}^{D \times d}, \quad W^T W = I_d$$

Project: $y_i = W^T (x_i - \bar{x}) \in \mathbb{R}^d$
Reconstruct: $\hat{x}_i = \bar{x} + W y_i$

Choose d

Retain $\geq 90\%$ variance: $\sum_{j=1}^d \lambda_j \geq 0.9 \sum_{k=1}^D \lambda_k$

PCA minimizes reconstruction error

$$e_i = \|\hat{x}_i - x_i\|^2$$

The d leading eigenvectors minimize total reconstruction error.

Limitation: only captures linear structure.

Kernel PCA

When: non-linear structure in data.
Map $x_i \mapsto \phi(x_i)$, apply PCA in feature space. By representer theorem, eigenvectors of covariance lie in span of $\{\phi(x_i)\}$, so:

$$K a = \lambda N a, \quad K_{ij} = k(x_i, x_j)$$

$$y_i^{(j)} = \sum_{l=1}^N a_l^{(j)} k(x_i, x_l)$$

Center kernel: $\tilde{K} = K - \mathbf{1}_N K - K \mathbf{1}_N + \mathbf{1}_N K \mathbf{1}_N$
Limitation vs PCA: no inverse mapping (can't reconstruct $\hat{x}$ from y).

Autoencoders

When: non-linear structure; need encoder & decoder; large data.

Autoencoder

Encoder: $x \rightarrow y$ (latent representation)
Decoder: $y \rightarrow \hat{x}$
Goal: $\hat{x} \approx x$, trained by minimizing $\frac{1}{N} \sum_i \|\hat{x}_i - x_i\|^2$ with SGD.

Variants

Denoising: add noise to input, train to reconstruct clean version Sparse: add ℓ_1 penalty on latent code $\Rightarrow$ sparse activations

vs PCA: autoencoder is non-linear, more powerful; has both encoder & decoder; non-convex optimization.
Applications: compression, anomaly detection, generative models.

12 K-Means Clustering

When: unsupervised; want to partition N unlabeled points into K groups.

Algorithm

1. Initialize K centroids $\mu_1, \dots, \mu_K$ randomly.
2a. Assign: $c_i = \arg \min_k \|x_i - \mu_k\|^2$ for all i .
2b. Update: $\mu_k = \frac{1}{|C_k|} \sum_{i:c_i=k} x_i$.
3. Repeat until assignments don't change.

Key Caveats

Convergence: guaranteed, but may reach local minimum (not global) Initialization: run multiple times with different random inits Scale: always normalize features — K-Means uses Euclidean distance! Choose K : elbow method on within-cluster variance

Complexity: $O(NKD)$ per iteration.
Limitation: assumes spherical clusters; sensitive to outliers.

13 Overfitting, Underfitting & CV

Bias-Variance

Underfitting: high error on train & test (model too simple) Overfitting: low train error, high test error (model too complex)

Cross-Validation Strategies

Val. set: 80/20 split; fast but estimate depends on random split K-fold: split in K parts, rotate; $K=5$ or 10 is standard LOOCV: $K = N$; exact but very expensive

Rule: never use test set to choose hyperparameters — use validation/CV only.

14 Evaluation — Regression

Regression Metrics

MSE: $\frac{1}{N} \sum (y_i - \hat{y}_i)^2$ — penalizes large errors
RMSE: $\sqrt{\text{MSE}}$ — same unit as y ; more interpretable MAE: $\frac{1}{N} \sum |y_i - \hat{y}_i|$ — robust to outliers
MAPE: $\frac{1}{N} \sum \left| \frac{y_i - \hat{y}_i}{y_i} \right|$ — relative; undef. if $y_i=0$

Interpretation

MSE ≈ 0 : near-perfect predictions RMSE: error in the unit of y , e.g. RMSE=5€ means avg error of 5€ MAE vs MSE: MAE if outliers shouldn't dominate; MSE if large errors are critical

Confusion Matrix (binary)		
	True Pos	True Neg
Pred Pos	TP	FP (type I)
Pred Neg	FN (type II)	TN
$P = TP + FN \quad N = FP + TN$		

Metrics
Accuracy: $\frac{TP+TN}{TP+TN+FP+FN}$ — <i>misleading if imbalanced!</i> Precision: $\frac{TP}{TP+FP}$ — of all predicted pos, how many are truly pos Recall: $\frac{TP}{TP+FN}$ — of all true pos, how many were detected FPR: $\frac{FP}{FP+TN}$ — fraction of negatives falsely flagged F1: $2 \cdot \frac{P \cdot R}{P+R}$ — harmonic mean; use when imbalanced

Interpretation
High Precision: few false alarms (spam filter, recommend.) High Recall: few misses (cancer detection, fraud detection) F1 $\approx$ 1: excellent balance precision/recall F1 $\approx$ 0: poor model Precision vs Recall: trade-off: adjust threshold to prioritize one

- Multi-class:** $C \times C$ confusion matrix. Compute precision/recall per class (one-vs-all) then average.
- **Macro avg:** equal weight per class
 - **Weighted avg:** weight by class size; use if imbalanced

16 Model Comparison Summary			
Model	T	Train	Hyper
Lin. Reg.	R	closed form	—
Log. Reg.	C	GD	η
Softmax	C	GD	η
SVM	C	QP solver	C
KNN		R/C none	k
Ridge	R	closed form	λ
Kernel Ridge	R/C	closed form	λ, k
MLP		R/C SGD+backprop	η, L, M
PCA	DR	eigendecomp	d
Kernel PCA	DR	eigendecomp	d, k
Autoencoder	DR	SGD	arch
K-Means	CL	iterative	K

Model	Key trait
Lin. Reg.	closed form $w^* = (X^T X)^{-1} X^T y$; linear only
Log. Reg.	sigmoid output; probabilistic; no closed form
Softmax	generalizes logistic to C classes
SVM	max-margin; C large $\rightarrow$ tight (overfit)
KNN	no training; slow inference; normalize!
Ridge	L2 penalty $\lambda \ w\ ^2$; always invertible
Kernel Ridge	implicit ϕ ; $\alpha^* = (K + \lambda I)^{-1} y$
MLP	learns ϕ automatically; non-convex
PCA	linear; max variance; has reconstruction
Kernel PCA	non-linear; no inverse (no reconstruction)
Autoencoder	non-linear; has encoder & decoder
K-Means	unsupervised; local min; normalize!
R=Reg.	C=Classif. DR=Dim. Red. CL=Clustering